

Chebyshev Collocation, Smolyak Sparse Grids, and Two-Branch Quadrature

A Textbook Companion to the Global Solver in `code/bocola2016/`

Computational methods note — Bocola (2016) replication package

August 10, 2026

Abstract

This note explains, at a graduate textbook level, the numerical machinery that the `code/bocola2016/` package uses to solve a recursive competitive equilibrium with priced sovereign default risk. Three ideas do the work. **Chebyshev polynomial approximation** turns an unknown policy function into a short vector of coefficients with near-optimal, spectrally accurate interpolation on carefully chosen nodes. **Smolyak sparse grids** extend that construction to six state variables without paying the tensor-product “curse of dimensionality.” And a **two-branch Gauss–Hermite quadrature** evaluates the conditional expectations inside the agents’ Euler equations, splitting the future into a no-default and a default continuation weighted by a priced probability. For each idea we give the underlying theory, the specific numerical hazards it carries, and the exact place in the code where it lives. The recurring theme is that every one of these tools is a controlled approximation whose error is well understood only inside a region the model actually visits—and that respecting that region (clipping, rotation, smoothing, best-iterate tracking) is what separates a working global solver from a diverging one.

Contents

1	The problem being solved	2
2	Univariate Chebyshev approximation	3
2.1	Chebyshev polynomials of the first kind	3
2.2	The nodes: why not equispaced?	3
2.3	The Lebesgue constant: near-optimality	4
2.4	Convergence rate: spectral accuracy	5
2.5	Collocation as a linear solve	6
3	Smolyak sparse grids	6
3.1	The tensor-product curse	6
3.2	Nested one-dimensional levels	6
3.3	The Smolyak combination rule	7
3.4	What Smolyak approximates, and anisotropy	8
3.5	The PCA-rotated grid	8
4	Numerical risks in the computation	8
4.1	Extrapolation blow-up	9
4.2	Conditioning and the sparse-grid Lebesgue constant	9
4.3	Aliasing and non-interpolatory error	9
4.4	The residual curse of dimensionality	9
4.5	Non-smoothness: kinks, Gibbs, and Fischer–Burmeister	9
4.6	Boundary infeasibility and near-unit-root states	10
4.7	Interaction with the outer fixed point	10

5	The branch computation	11
5.1	The economic object	11
5.2	Gauss–Hermite quadrature over the innovations	11
5.3	Splitting into branches	12
5.4	What differs across branches	12
5.5	From branch pieces to prices and the multiplier	13
5.6	Why the branch structure is the whole point	13
6	Assembling the solver	14
7	Summary	14
	Where to read the code	15
	References and further reading	15

1 The problem being solved

The model is a detrended recursive competitive equilibrium. The aggregate state is the six-vector

$$S = (\underbrace{K, B, P}_{\text{endogenous}}, \underbrace{\Delta z, g, s}_{\text{exogenous}}) \in \mathbb{R}^6,$$

capital K , the sovereign bond stock B , banks’ promised deposit payment P , the technology growth innovation Δz , the fiscal shock g , and the *risk factor* s that governs the priced probability of a sovereign default next period. A solution is not a number or a finite path; it is a set of *decision rules*—functions of the state—for consumption $C(S)$, the deposit rate $R(S)$, the bond price $Q_B(S)$, and the banker’s marginal value of wealth $\alpha(S)$ (written `av` in the code). These functions must satisfy, at *every* point of the state space, a system of functional equations: three Euler equations (deposits, capital, sovereign bonds), the recursion for $\alpha(S)$, and an occasionally-binding leverage constraint.

Two facts force a *global, nonlinear* method rather than a local (perturbation) one. First, the leverage constraint binds only occasionally—it is slack in normal times and tightens sharply in stress—so the policy functions have a *kink* that a Taylor expansion around the balanced-growth path cannot capture. Second, the object of interest is precisely the behaviour far from that path (a sovereign-risk crisis). A global method represents each unknown function by a finite basis expansion and pins down the coefficients by forcing the residual of the functional equations to zero at a finite set of *collocation nodes*. This is the *projection* (or *weighted-residual*) approach. Three questions organize the rest of the note:

1. **Which basis and which nodes?** → Chebyshev polynomials on Chebyshev–Lobatto nodes (§2).
2. **How to do this in six dimensions?** → Smolyak sparse grids (§3).
3. **How to evaluate the expectations that make the equations *Euler* equations?** → two-branch Gauss–Hermite quadrature (§5).

Section 4 is devoted entirely to the numerical *risks*—the ways each tool fails, and how the code defends against them.

2 Univariate Chebyshev approximation

Everything multidimensional in this code is built from one univariate object, so we start there. The goal is to approximate a smooth function $f : [-1, 1] \rightarrow \mathbb{R}$ by a polynomial p_n of degree n such that (i) the error is as small as possible, (ii) the map from function values to coefficients is well conditioned, and (iii) both are cheap to evaluate.

2.1 Chebyshev polynomials of the first kind

Definition 1. *The Chebyshev polynomials of the first kind are defined on $[-1, 1]$ by*

$$T_k(x) = \cos(k \arccos x), \quad k = 0, 1, 2, \dots$$

The definition looks trigonometric but T_k is a genuine polynomial of degree k : with $x = \cos \theta$, $\cos(k\theta)$ expands into powers of $\cos \theta$. Equivalently, and this is exactly how the code computes them, they obey the three-term recurrence

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_k(x) = 2x T_{k-1}(x) - T_{k-2}(x). \quad (1)$$

This recurrence is the entire content of `chebyshev_basis_1d` in `smolyak.py`:

```
def chebyshev_basis_1d(x, max_deg):
    T = np.empty((x.size, max_deg + 1))
    T[:, 0] = 1.0
    if max_deg >= 1:
        T[:, 1] = x
    for k in range(2, max_deg + 1):
        T[:, k] = 2.0 * x * T[:, k - 1] - T[:, k - 2]
    return T
```

The recurrence is used rather than the monomial form $\sum a_k x^k$ because the monomial (Vandermonde) representation is catastrophically ill-conditioned—the basis functions x^k become nearly linearly dependent as k grows—whereas (1) is numerically stable.

Proposition 1 (Orthogonality). *Under the weight $w(x) = (1 - x^2)^{-1/2}$,*

$$\int_{-1}^1 \frac{T_j(x) T_k(x)}{\sqrt{1 - x^2}} dx = \begin{cases} 0 & j \neq k, \\ \pi & j = k = 0, \\ \pi/2 & j = k \geq 1. \end{cases}$$

This is the change of variables $x = \cos \theta$ applied to the elementary orthogonality of $\{\cos k\theta\}$ on $[0, \pi]$. Orthogonality is what makes a Chebyshev expansion a *cosine series in disguise*: approximating $f(x)$ by $\sum_k c_k T_k(x)$ is the same as approximating the even, 2π -periodic function $f(\cos \theta)$ by a Fourier cosine series. All the good behaviour of Fourier series on periodic functions transfers, without their bad behaviour at non-periodic endpoints.

2.2 The nodes: why not equispaced?

A degree- n interpolant is pinned by $n + 1$ nodes. The naive choice—evenly spaced points—is a disaster for polynomial interpolation. The classic counterexample is Runge’s function $f(x) = 1/(1 + 25x^2)$: as the degree rises, the equispaced interpolant develops ever-larger oscillations near $x = \pm 1$ and *diverges* in the sup norm, even though f is perfectly smooth (Figure 1, left). This is the *Runge phenomenon*, and it reflects a deep fact: interpolation on equispaced nodes is exponentially ill-conditioned.¹

¹See the discussion of Runge/Lebesgue in Trefethen’s *Approximation Theory and Approximation Practice*, and the survey references collected at the end of this note.

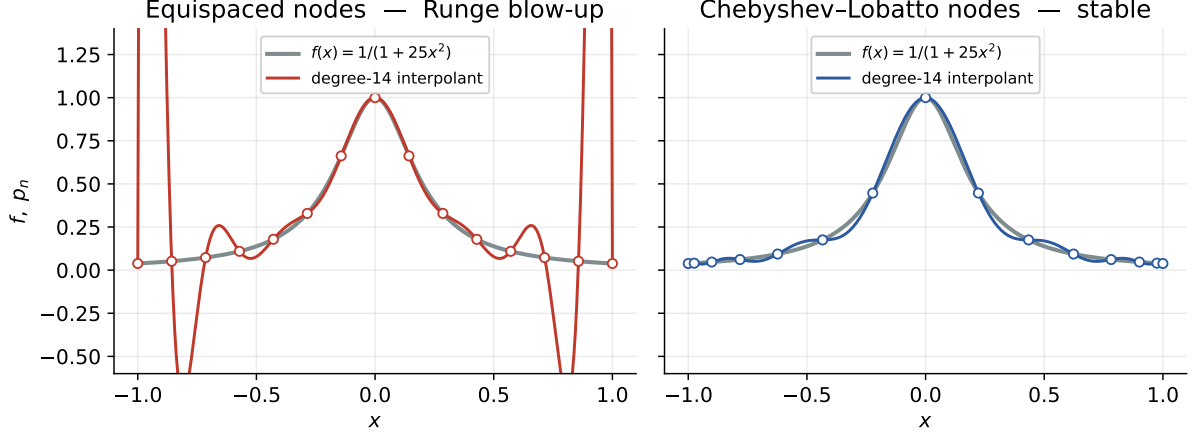


Figure 1: Degree-14 polynomial interpolation of Runge’s function. Equispaced nodes (left) oscillate wildly near the endpoints; Chebyshev–Lobatto nodes (right) track the function. Same degree, same function—only the node placement differs.

The cure is to cluster nodes toward the endpoints. The *Chebyshev extrema*, also called *Chebyshev–Lobatto* or *Clenshaw–Curtis* points, do exactly this:

$$x_k = -\cos\left(\frac{\pi k}{m-1}\right), \quad k = 0, \dots, m-1, \quad (2)$$

which are the projections onto the x -axis of m equally spaced points on the upper unit semicircle (Figure 2). This is the `level_points` routine in `smolyak.py`. Their density near ± 1 scales like $(1-x^2)^{-1/2}$ —exactly the Chebyshev weight—so the interpolation problem becomes well conditioned.

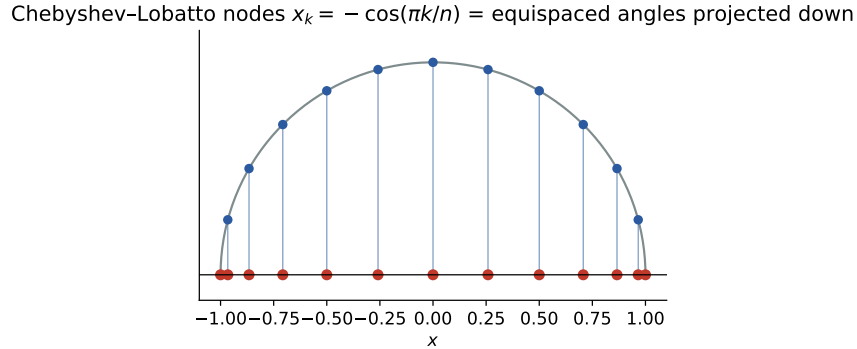


Figure 2: The Chebyshev–Lobatto nodes (2) are equally spaced *angles* on the semicircle projected down onto $[-1, 1]$; the projection clusters them near the endpoints, which is what tames the Runge phenomenon.

2.3 The Lebesgue constant: near-optimality

The quality of a set of nodes is measured by the *Lebesgue constant* Λ_n , the operator norm of the interpolation map from $C[-1, 1]$ to itself. It bounds how far interpolation can be from the best possible polynomial approximation p_n^* :

$$\|f - p_n\|_\infty \leq (1 + \Lambda_n) \|f - p_n^*\|_\infty. \quad (3)$$

A small Λ_n means “interpolating is almost as good as doing the best you possibly could.” The contrast is stark:

$$\Lambda_n^{\text{equispaced}} \sim \frac{2^{n+1}}{e n \log n} \quad (\text{exponential}), \quad \Lambda_n^{\text{Chebyshev}} \sim \frac{2}{\pi} \log n \quad (\text{logarithmic}).$$

Chebyshev nodes are not *exactly* optimal—the minimizing nodes have no closed form—but they capture essentially all of the available accuracy, with a Lebesgue constant that grows so slowly ($\log n$) that it is irrelevant in practice. This is the precise sense of the phrase “Chebyshev interpolation is near-minimax.”

2.4 Convergence rate: spectral accuracy

How fast does the error fall as we add degrees? The answer depends entirely on the smoothness of f :

Theorem 1 (Rate governed by smoothness). *Let p_n be the degree- n Chebyshev interpolant of f on $[-1, 1]$.*

- *If f has ν continuous derivatives with bounded variation, then $\|f - p_n\|_\infty = O(n^{-\nu})$ (algebraic).*
- *If f is analytic in a Bernstein ellipse with parameter $\rho > 1$ around $[-1, 1]$, then $\|f - p_n\|_\infty = O(\rho^{-n})$ (geometric / spectral).*

Figure 3 shows both regimes on Chebyshev nodes: the analytic target $e^{\sin 3x}$ hits machine precision by degree ~ 30 (straight line on a log scale = geometric decay), while the kinked target $|x - 0.2|$ crawls down algebraically. This dichotomy is the single most important fact for the model at hand. Where its policy functions are smooth, a handful of Chebyshev degrees delivers near-machine accuracy. But the occasionally-binding leverage constraint injects a kink into the true policy, and *at that kink the spectral rate collapses to algebraic*—a polynomial basis chasing a non-smooth target produces Gibbs-type ringing. This is not a coding defect; it is a theorem about polynomials, and it motivates the smoothing devices of §4.

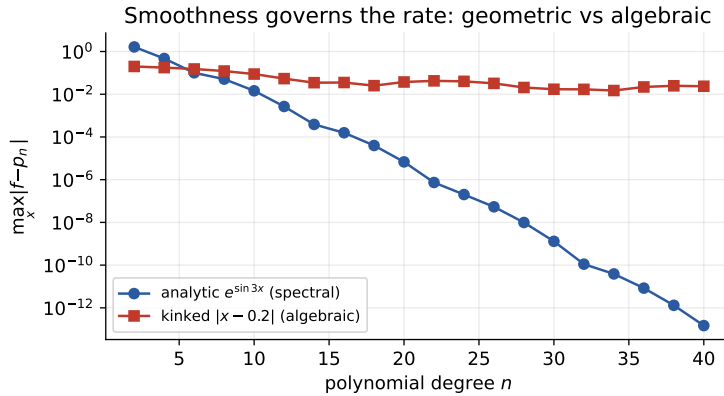


Figure 3: Maximum interpolation error versus polynomial degree on Chebyshev nodes. Analytic functions converge geometrically (spectral accuracy); a kink forces algebraic convergence. The model’s occasionally-binding constraint puts a kink in the policy, so the second curve is the relevant warning.

2.5 Collocation as a linear solve

Fix the degrees $\{0, 1, \dots, n\}$ and the nodes $\{x_k\}_{k=0}^n$. Writing the interpolant as $p(x) = \sum_j c_j T_j(x)$ and demanding $p(x_k) = f(x_k)$ at every node gives the square linear system

$$\underbrace{[T_j(x_k)]_{k,j}}_{\text{basis matrix } \Phi} c = f, \quad (4)$$

so the coefficients are $c = \Phi^{-1}f$. Because Φ depends only on the grid, not on f , the code factorizes it *once* (an LU factorization) and reuses it every time it needs coefficients from values. In `SmolyakGrid` this is the pair `fit` (solve for c) and `eval` (form $\Phi(x)c$ at arbitrary points), with the LU factor `self.lu` built in the constructor. Two consequences worth internalizing:

- **Interpolation is exact at the nodes by construction.** The interpolant reproduces f at every collocation point to rounding error; all approximation error lives *between* nodes.
- **Evaluating the interpolant outside $[-1, 1]$ is extrapolation**, and Chebyshev extrapolation grows like $|T_n(x)| \sim |x + \sqrt{x^2 - 1}|^n$ —explosively. This is the single most dangerous property of the whole scheme and is treated in §4.1.

Finally, the code maps the model’s natural state box $[\ell, h]$ onto the reference interval by the affine transform $u = 2(x - \ell)/(h - \ell) - 1$ (`to_unit/from_unit`); all Chebyshev machinery lives in the $u \in [-1, 1]$ coordinates.

3 Smolyak sparse grids

Section 2 solved the one-dimensional problem. The model has six state dimensions. The obvious extension—a *tensor product* of one-dimensional grids—is computationally fatal, and the fix is Smolyak’s 1963 construction, brought into economics by Krueger and Kubler (2004) and refined by Judd, Maliar, Maliar and Valero (2014).

3.1 The tensor-product curse

A tensor grid with m points per dimension has m^d points in d dimensions. With a modest $m = 5$ and the model’s $d = 6$ that is $5^6 = 15,625$ collocation nodes—and every node requires solving the agents’ nonlinear system and a full expectation, at every outer iteration. At $m = 9$ it is over half a million. The count grows *geometrically in the dimension*: this is Bellman’s *curse of dimensionality* (Figure 4, grey line). Tensor grids are simply not an option past three or four states.

3.2 Nested one-dimensional levels

The Smolyak idea rests on a sequence of *nested* one-dimensional grids. Define level i to have m_i Chebyshev–Lobatto points, with

$$m_1 = 1, \quad m_i = 2^{i-1} + 1 \quad (i \geq 2) \Rightarrow m = 1, 3, 5, 9, 17, \dots \quad (5)$$

The doubling rule (5) makes the grids *nested*: every point of level i is also a point of level $i+1$ (`_level_points`). Nestedness is what lets Smolyak add resolution without recomputing anything, and it is why Clenshaw–Curtis/Chebyshev extrema (rather than, say, Gauss points) are the natural building block. The code works with the *incremental* sets—the points and the polynomial degrees that are *new* at level i :

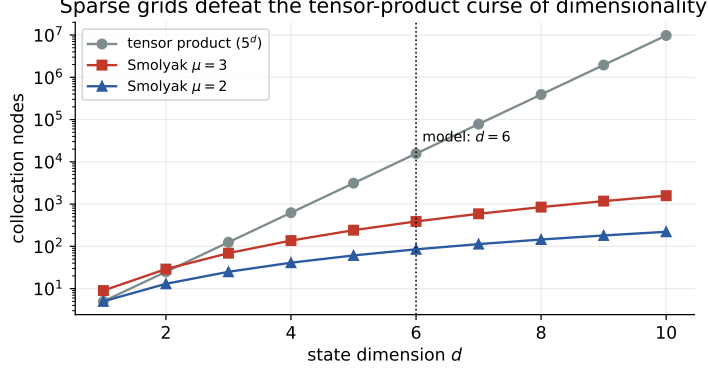


Figure 4: Node count versus state dimension. A tensor product explodes geometrically; the Smolyak grids at levels $\mu = 2, 3$ grow polynomially. At the model’s $d = 6$ the Smolyak grid has 85 ($\mu=2$) or 389 ($\mu=3$) nodes—versus tens of thousands for a comparable tensor grid.

```
def _new_points(i):          # points introduced at level i (disjoint across i)
    if i == 1: return np.array([0.0])
    if i == 2: return np.array([-1.0, 1.0])
    return _level_points(i)[1::2]      # odd positions absent from level i-1

def _new_degrees(i):        # Chebyshev degrees introduced at level i
    if i == 1: return np.array([0])
    if i == 2: return np.array([1, 2])
    return np.arange(2**(i-2)+1, 2**(i-1)+1)
```

Crucially $|\text{new points}| = |\text{new degrees}|$ at every level. This is the bookkeeping that keeps the eventual multidimensional basis matrix *square*.

3.3 The Smolyak combination rule

In d dimensions, index a tensor block by a multi-index $\mathbf{i} = (i_1, \dots, i_d)$ of per-dimension levels. A full tensor grid at level $\mu+1$ would take *all* blocks with each $i_j \leq \mu+1$. Smolyak instead keeps only the blocks whose *total* level is bounded:

$$\text{keep } \mathbf{i} \iff \sum_{j=1}^d (i_j - 1) \leq \mu, \quad (6)$$

where $\mu \geq 0$ is the *approximation level*. The grid is the union of the tensor products of the *new-point* sets over all kept $\mathbf{i}$; the basis is the union of the tensor products of the *new-degree* sets over the same $\mathbf{i}$. This is precisely `_multi_indices` + the assembly loop in `SmolyakGrid.__init__`:

```
for i_vec in _multi_indices(d, mu, mu_vec):
    axes_p = [_new_points(i) for i in i_vec]      # nested points
    axes_d = [_new_degrees(i) for i in i_vec]     # matching degrees
    pts.append(cartesian_product(axes_p))         # tensor of the *new* sets
    degs.append(cartesian_product(axes_d))
```

Because points and degrees are added in lock-step, the assembled basis matrix $\Phi = [\prod_j T_{\deg_j}(u_j)]$ evaluated at the grid points is *square and invertible*: the interpolant is *exact at every Smolyak node*, and `fit/eval` are again a single LU solve. Selecting only “low total level” blocks is exactly what drops the expensive high-resolution-in-many-dimensions-at-once corners that a tensor grid wastes its budget on. In two dimensions the effect is visible directly (Figure 5): the Smolyak grid is a sparse cross-shaped subset of the full tensor grid.

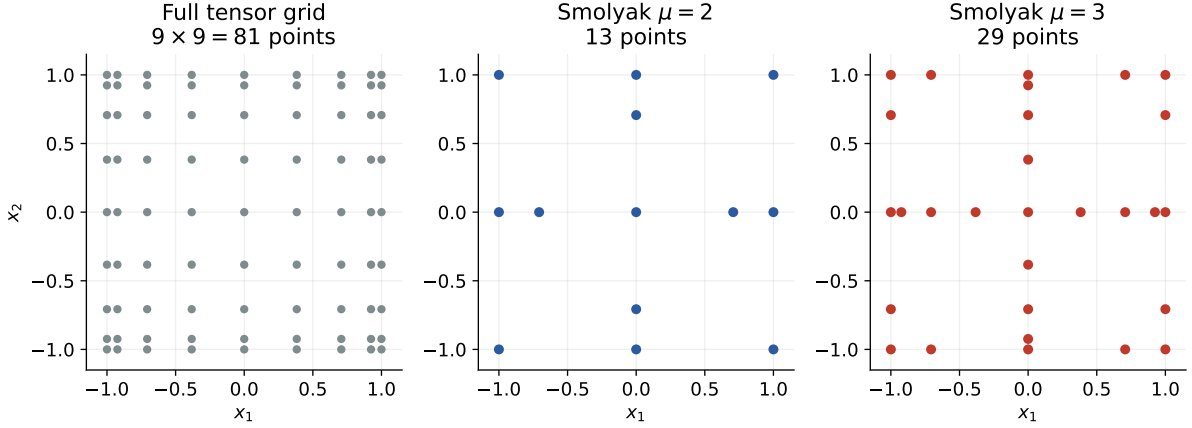


Figure 5: In two dimensions: the full 9×9 tensor grid (left) versus the Smolyak grids at $\mu = 2$ (13 nodes) and $\mu = 3$ (29 nodes). The sparse grids keep the axes and low-order cross terms and discard the dense interior—the source of the savings.

3.4 What Smolyak approximates, and anisotropy

The resulting interpolant is exact for the *hyperbolic-cross* polynomial space—all monomials $\prod_j x_j^{a_j}$ with a bounded weighted total degree—rather than the full tensor space. Practically, $\mu = 2$ captures all quadratic cross terms (a complete second-order approximation plus selected higher terms); $\mu = 3$ adds cubic interactions. The package’s gate tests confirm exactly this: on-grid exactness to 10^{-10} , complete quadratic exactness including cross terms, and geometric error decay $10^{-1} \rightarrow 10^{-2} \rightarrow 5 \times 10^{-4}$ as μ runs 2, 3, 4 on a smooth 6-D test function.

The code also supports *anisotropic* levels through `mu_vec`: a per-dimension cap $i_j - 1 \leq \mu_j$ in `multi_indices`. This is the Judd–Maliar–Valero refinement—spend resolution on the dimensions where the policy actually curves and economize on the rest. The model does not lean on anisotropy heavily, but the hook is there.

3.5 The PCA-rotated grid

One more construction deserves mention because it is where sparse-grid practice meets the specific economics. A box aligned with the physical axes (K, B, P) badly overcovers the state space: the ergodic cloud is elongated along a diagonal (more capital $\Rightarrow$ more deposits $\Rightarrow$ higher obligations), so an axis-aligned box has deeply *infeasible* anti-correlated corners (low assets, high obligations) that the model never visits but that the collocation fit is nonetheless forced to represent. `RotatedGrid` (`rotated_grid.py`) diagonalizes the ergodic covariance of (K, B, P), builds the Smolyak box in the principal-component coordinates, and presents a physical-coordinate interface so the rest of the code is unchanged. Rotating cut the infeasible-corner fraction from $\sim 48\%$ to $\sim 16\%$ and delivered machine precision on-grid. It is a concrete instance of the general lesson in §4: a global polynomial basis is only as good as the region you ask it to cover.

4 Numerical risks in the computation

Chebyshev–Smolyak collocation is powerful but not forgiving. This section catalogues the ways it goes wrong and the specific defenses in the code. A PhD reader should treat this as the “what will bite you” section.

4.1 Extrapolation blow-up

The gravest hazard. Outside the box, Chebyshev polynomials grow like ρ^n ; a rule fitted on $[-1, 1]$ and evaluated at $u = 1.2$ can return a *negative* consumption or a nonsensical bond price, which then feeds back through the expectation and produces NaNs. The danger is acute here because the expectation step evaluates next-period rules at *stochastically generated* future states that can wander outside the current box. The defense is to *clip* the evaluation point back into the box before every interpolation:

```
S_eval = grid.clip(S_next)          # never extrapolate the Chebyshev rule
Cn = np.maximum(grid.eval(coef["C"], S_eval), 1e-8)
QBn = np.maximum(grid.eval(coef["QB"], S_eval), 1e-4)
```

Clipping is a *documented, deliberate* projection (`clip` is a public method, not a silent guard), and the numeric floors (10^{-8} , 10^{-4}) are a second line of defense against the residual overshoot near the boundary. The cost is a small bias when the true state genuinely leaves the box; the package controls it by choosing the box wide enough that the ergodic path is 99.3% interior, so clipping essentially never binds on a simulated path.

4.2 Conditioning and the sparse-grid Lebesgue constant

The one-dimensional Lebesgue constant is a gentle $\log n$ (§2.3), but on Smolyak grids it grows faster—roughly $(\log n)^d$ -like behaviour in d dimensions—so the amplification of interpolation error, and the conditioning of the basis matrix Φ , degrade as the level μ or the dimension rises. In practice this is why the code lives at $\mu = 2$ (occasionally $\mu = 3$): high levels buy formal accuracy but a worse-conditioned solve. The stable Chebyshev *recurrence* (not the monomial basis) and the once-and-for-all LU factorization keep the conditioning as good as the node set allows, but they cannot repair an intrinsically hard node set—hence the preference for modest levels plus a well-placed box over brute-force refinement.

4.3 Aliasing and non-interpolatory error

A sparse grid is exact only on the hyperbolic-cross polynomial space. Any content of the true policy outside that space is *aliased*—misattributed to the retained basis functions. For a smooth policy this is negligible (the aliased content is tiny), but two situations amplify it: (i) high-frequency features from a near-kink, and (ii) using the same sparse construction for *integration*, where the Smolyak/Clenshaw–Curtis weights can be *negative*, so a sparse quadrature of a sharply peaked integrand can return a value outside the range of the integrand. The code sidesteps (ii) entirely by *not* using the sparse grid to integrate: expectations are done with a dedicated Gauss–Hermite tensor rule (§5), whose weights are strictly positive.

4.4 The residual curse of dimensionality

Smolyak defeats the *geometric* curse but not all cost growth. The node count still rises polynomially in d (Figure 4), and Krueger–Kubler and Malin–Krueger–Kubler document that beyond roughly twenty state variables even sparse grids become expensive. Six states is comfortably inside the sweet spot; the point for the reader is that “Smolyak breaks the curse” is a statement about the *exponent*, not a claim of free lunch.

4.5 Non-smoothness: kinks, Gibbs, and Fischer–Burmeister

This is the model-specific hazard. The occasionally-binding leverage constraint,

$$\mu = \max\left\{1 - \frac{\mathbb{E}[\hat{\Lambda}'] R N}{\lambda(Q_K K' + Q_B B')}, 0\right\},$$

puts a max-kink into the policy. Two distinct problems follow. First, as a matter of *approximation theory*, a global polynomial basis converges only algebraically at a kink and produces Gibbs oscillations nearby (§2.4)—the sparse grid *cannot* represent the constraint boundary sharply. Second, as a matter of *solving*, the max makes the finite-difference Jacobian of the outer non-linear solve noisy and stalls Newton. The code’s answer, imported from the two-country solver, is *Fischer–Burmeister* complementarity smoothing: make μ an explicit unknown and replace the max with the equation

$$\varphi(\mu, \text{slack}) = \mu + \text{slack} - \sqrt{\mu^2 + \text{slack}^2} = 0, \quad (7)$$

which is zero exactly when $\mu \geq 0$, $\text{slack} \geq 0$, and $\mu \cdot \text{slack} = 0$ —the complementarity conditions—and is smooth everywhere except the single point $(0, 0)$ (`_fb` in `newton_collocation.py`). A companion trick smooths a second kink *inside* the expectation: the hard investment floor $\mathbf{1}\{I' > 0\}$ is replaced by a softplus that equals I' for feasible investment to 10^{-12} and bends smoothly to a floor only near $I' = 0$, so the Jacobian stays well-defined at the quadrature nodes. The general principle—any new max / min / $\mathbf{1}\{\cdot\}$ on a Newton-solved quantity needs FB-style smoothing or the finite-difference Jacobian stalls—is a recurring rule in this codebase.

4.6 Boundary infeasibility and near-unit-root states

The model’s capital has a near-unit-root (~ 0.98) dynamic, so the ergodic set is *wide*. A box wide enough to contain it necessarily includes corner states that are economically infeasible (negative net worth, $\mu \rightarrow 1$, the banker value function undefined). There, the pointwise non-linear solve has no sensible solution, and if those garbage values enter the global fit they poison the interpolant *everywhere* through the coupling of the collocation system. The code deploys three countermeasures, each addressing one facet:

- **Anchoring** (`make_residual`, `feasible_mask`): infeasible corners are held at their warm-start values via an anchor residual $V - \text{target}$ so the square system stays well conditioned while Newton polishes the feasible interior.
- **Rejection** (`MU_ACCEPT`): a pointwise solution whose implied μ exceeds a physical threshold is rejected and the prior value kept, so a spurious “maximal-binding” trial cannot corrupt the fit. (The threshold is 0.10 in the no-default model where μ is truly tiny, raised to 0.85 in the default regime where a 55% haircut makes the constraint bind hard for real.)
- **Rotation** (§3.5): reshaping the box to the ergodic cloud removes most infeasible corners at the source.

The honest limitation, documented in the replication notes, is that these mitigate but do not eliminate the problem: a residual $\sim 40\text{--}60\%$ of axis-aligned corners must be anchored, and their frozen values floor the achievable residual of the *global* fit. This is intrinsic to a *global* polynomial basis on a wide box—corner errors are non-local—and is the reason the notes flag a local/finite-element basis as the principled next step.

4.7 Interaction with the outer fixed point

Finally, the approximation error interacts with the solution algorithm. Time iteration is successive approximation on the policy; its contraction rate equals the model’s persistence (~ 0.98), so it crawls and can *oscillate*—reaching a good near-fixed-point and then drifting away, because the two-regime near-unit-root iteration is not globally contractive. The code’s defenses are outer *damping* (update $V \leftarrow (1 - \text{damp})V + \text{damp } V^{\text{new}}$), convergence measured on rule *values* rather than coefficients (the kink makes coefficients cycle even when values settle), and *best-iterate*

tracking in `time_iterate_full` (keep the iterate with the fewest infeasible points and smallest update, not the last). The alternative Newton solver on the stacked collocation residual (`newton_collocation.py`) trades the eigenvalue-limited linear rate for quadratic convergence, at the cost of the kink-handling above.

5 The branch computation

We now come to the piece the reader specifically asked about: how the code computes “the branch.” The term refers to the *two-branch quadrature* that evaluates the conditional expectations inside the agents’ Euler equations, splitting next period into a no-default branch and a default branch. Conceptually this is where the priced sovereign risk enters the model, and it is the reason a *global* solution was built in the first place.

5.1 The economic object

An Euler equation is an equilibrium condition of the form “price today = expected discounted payoff tomorrow.” The expectation is over next period’s shocks *and* over whether the sovereign defaults. Formally, for any payoff X' the banker’s/household’s condition needs

$$\mathbb{E}_t[X'] = \iiint \left[(1 - p^d) X'^{(d'=0)} + p^d X'^{(d'=1)} \right] d\Phi(\epsilon_z) d\Phi(\epsilon_g) d\Phi(\epsilon_s), \quad (8)$$

with two ingredients to approximate: the integral over the three Gaussian innovations $(\epsilon_z, \epsilon_g, \epsilon_s)$, and the discrete default lottery $d' \in \{0, 1\}$. Bocola’s key modeling point—and the reason the representative-branch shortcut in the sister `code/global/` model failed—is that this is a genuine *distribution over a discrete future event*, not a point mass. Sovereign risk raises the *probability weight* p^d on a low-payoff branch; it does not merely shift a mean.

5.2 Gauss–Hermite quadrature over the innovations

The three innovations are independent standard normals, so the integral over them is done by *Gauss–Hermite* quadrature—the exact tool for Gaussian weights. An n -node Gauss–Hermite rule integrates polynomials up to degree $2n - 1$ exactly against the normal density; the code uses the probabilists’ variant `hermegauss` and tensorizes it over the three shocks:

```
def gauss_hermite_3d(n=3):
    x, w = np.polynomial.hermite_e.hermegauss(n)    # E[f], eps ~ N(0,1)
    w = w / w.sum()
    xz, xg, xs = np.meshgrid(x, x, x, indexing="ij")
    wz, wg, ws = np.meshgrid(w, w, w, indexing="ij")
    return xz.ravel(), xg.ravel(), xs.ravel(), (wz*wg*ws).ravel()
```

With $n = 3$ nodes per shock this is $3^3 = 27$ quadrature nodes—and, note, this is a *tensor* rule with strictly positive weights, deliberately kept separate from the Smolyak grid (§4). Three nodes suffice because the integrand, once the policy rules are smooth, is well approximated by a low-degree polynomial in the shocks; the AR(1) laws of motion map each quadrature node to a next-period exogenous state:

$$\Delta z' = (1 - \rho_z)\gamma + \rho_z \Delta z + \sigma_z \epsilon_z, \quad s' = (1 - \rho_s)s^* + \rho_s s + \sigma_s \epsilon_s, \dots \quad (9)$$

(`next_exog_full`).

5.3 Splitting into branches

The default lottery is the discrete part of (8). The priced probability of default next period is a *logistic* function of the risk factor (Bocola’s equation 11):

$$p^d(s) = \frac{1}{1 + e^{-s}}. \quad (10)$$

The code computes the future integrand *once per branch* and then takes the probability-weighted combination. The heart of it is `expect_pieces_full` in `expectations_full.py`:

```
pd = default_prob(s)                                # weight on the default branch
inv0, lh0, rk0, rb0 = _branch_pieces(S_next, 0, grid, coef, cal) # d'=0
inv1, lh1, rk1, rb1 = _branch_pieces(S_next, 1, grid, coef, cal) # d'=1
w0, w1 = (1.0 - pd) * wq, pd * wq                  # branch x quadrature weights
return (np.dot(w0, inv0) + np.dot(w1, inv1),         # E[1/C']
        np.dot(w0, lh0) + np.dot(w1, lh1),          # E[w'/C']
        np.dot(w0, rk0) + np.dot(w1, rk1),          # capital-return integrand
        np.dot(w0, rb0) + np.dot(w1, rb1))          # bond-return integrand
```

Read this carefully, because it is the whole mechanism. `w0` and `w1` are outer products of the Gauss–Hermite weights `wq` with the scalar branch probabilities $1 - p^d$ and p^d : each of the 27 shock nodes is duplicated into a no-default copy and a default copy, and the two weight vectors sum to the full expectation. The four returned numbers are exactly the conditional-expectation “pieces” that the Euler residuals need— $\mathbb{E}[1/C']$, $\mathbb{E}[w'/C']$, and the capital- and bond-return integrands— evaluated over the joint distribution of shocks *and* default.

5.4 What differs across branches

The two branches share the same next-period *shock* nodes but evaluate different objects. `_branch_pieces` makes the distinction precise:

```
def _branch_pieces(S_next, d_next, grid, coef, cal):
    S_eval = grid.clip(S_next)                        # never extrapolate
    Cn = np.maximum(grid.eval(coef["C"][d_next], S_eval), 1e-8)
    QBn = np.maximum(grid.eval(coef["QB"][d_next], S_eval), 1e-4)
    ...
    surv = 1.0 - d_next * cal["haircut"]              # 1 if d'=0, 1-D if d'=1
    RBpay = surv * (pi + (1-pi)*(iota + QBn))         # haircut bond payoff
    ...
```

Three things branch:

1. **The rules used.** Decision rules carry a *separate coefficient vector per regime*, `coef[rule]` = `(coef_d0, coef_d1)`. Next period’s policy is read from whichever regime d' realizes— so the default branch continuation uses the crisis-regime rules, the no-default branch the normal-regime rules.
2. **The bond payoff.** In the default branch the survival share is $\text{surv} = 1 - D$ with $D = 0.55$ (the Greek PSI haircut), so the bond pays only 45% of its promise. This haircut, weighted by $p^d(s)$, is the direct channel by which higher priced risk lowers today’s bond price Q_B .
3. **The current-period statics.** The *current* regime d_{cur} (distinct from next period’s d') enters the current period map through the same `surv` factor—a haircut on bonds held *into* a default period.

Re-default is allowed: d' is drawn independently of the current regime, so both current regimes share the same two-branch continuation. This is the “distribution, not a point mass” structure spelled out in the file header.

5.5 From branch pieces to prices and the multiplier

The four expectation pieces close the model. Back in `point_block_full`, they are scaled by the discount factor $\beta C e^{-\Delta z}$ into $\mathbb{E}[\Lambda']$, $\mathbb{E}[\hat{\Lambda}']$ and the two return integrands, and combined into three unit-free Euler residuals plus the closed-form multiplier

$$\mu = \max \left\{ 1 - \frac{\mathbb{E}[\hat{\Lambda}'] R N}{\lambda(Q_K K' + Q_B B')}, 0 \right\} \quad (\text{mu_closed_form}), \quad (11)$$

which is Bocola’s Proposition-1 formula. The economic pass-through lives entirely in (10)–(11): a rise in the risk factor s raises p^d , which raises the weight `w1` on the haircut branch, which lowers the bond-return integrand `rb`, which lowers the equilibrium bond price Q_B that clears the bond Euler; the resulting mark-to-market loss on bank balance sheets lowers net worth N , which through (11) tightens the constraint ($\mu \uparrow$), which widens lending spreads. The replication reproduces exactly this signature: at Bocola’s 2.5% default probability the bond price falls -12% and net worth falls (Figure 6), the correct contractionary sign.

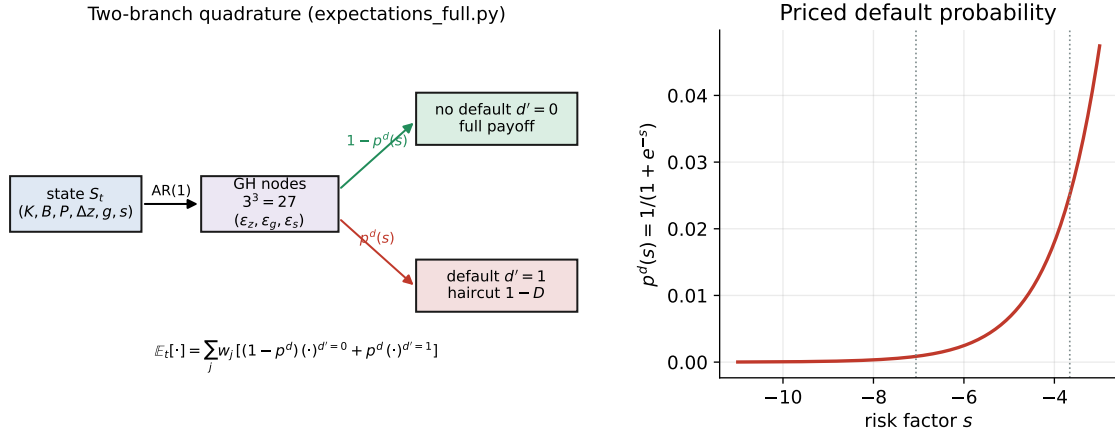


Figure 6: *Left*: the two-branch quadrature. The current state is pushed through the AR(1) laws to 27 Gauss–Hermite shock nodes, each of which splits into a no-default continuation (weight $1 - p^d$) and a haircut default continuation (weight p^d); the expectation is the probability- and quadrature-weighted sum. *Right*: the priced default probability $p^d(s) = 1/(1 + e^{-s})$ that sets the branch weights, from its balanced-growth value $s^* = -7.06$ up to the stress region.

5.6 Why the branch structure is the whole point

It is worth stating plainly why this machinery exists. The sister two-country model in `code/global/` approximates the same risk with a single *representative* post-default branch computed once and reused—a point-mass shortcut. That approximation gets the sign *wrong*: it makes the crisis-state bank lever *into* capital, so sovereign risk comes out expansionary. Bocola’s model has no separate “branch” to bolt on: the bad state is *the same decision rules evaluated at a different point of the state space*, reached with probability $p^d(s)$. Representing that faithfully requires (i) a global solution defined over the whole state space—hence Chebyshev–Smolyak—and (ii) the genuine two-branch quadrature above. The two halves of this note are therefore not independent: the sparse-grid interpolation exists precisely so that (8)–(10) can be evaluated at arbitrary future states, and the branch quadrature is what turns those interpolated rules into the priced-risk pass-through.

6 Assembling the solver

To place the pieces in the full loop, the solve is a fixed point over the rule coefficients:

1. **Seed.** Start from a first-order perturbation solution (a smooth, globally-roughly-correct guess), fitted onto the Smolyak grid. A constant-guess seed instead leaves a third of points infeasible.
2. **Inner solve.** At each of the 85 (or 389) grid points, and each regime $d \in \{0, 1\}$, solve the 3-equation Euler system for (C, R, Q_B) by bounded least squares, using the two-branch quadrature of §5 to form the expectations against the *current* coefficient vectors.
3. **Fit.** Convert the updated point values back to Chebyshev coefficients with the cached LU factor (`grid.fit`).
4. **Damp and iterate.** Damp the update, measure convergence on rule values, track the best iterate, and repeat until the update norm falls below tolerance. Optionally polish with the FB-smoothed Newton solver.
5. **Validate.** Simulate the solved rules forward to trace the ergodic set, and report the Euler residual *along the path the model actually visits*. The accepted accuracy standard—Bocola’s own—is accuracy on the ergodic set, which the package meets (mean $\log_{10}$ residual ≈ -3.1 , i.e. $\sim 0.1\%$), even though a wide-box global fit cannot be uniformly machine-accurate in the never-visited infeasible corners.

7 Summary

The computation rests on three theorems and one modeling insight. *Chebyshev interpolation* on Lobatto nodes is near-minimax and spectrally accurate for smooth functions—so a policy function becomes a short, stable coefficient vector—while being explosively wrong under extrapolation and only algebraically accurate at kinks. *Smolyak sparse grids* lift that construction to six dimensions at polynomial rather than geometric cost, by keeping only low-total-level tensor blocks of nested Chebyshev levels, at the price of a faster-growing Lebesgue constant and exactness on only the hyperbolic-cross polynomial space. *Gauss–Hermite quadrature* evaluates the Gaussian expectations exactly to high polynomial degree with positive weights. And the *two-branch structure* is the economic heart: sovereign risk is a probability weight $p^d(s)$ on a haircut-default continuation, evaluated by interpolating the crisis-regime rules at stochastically generated future states. The recurring discipline that makes all of this work is respect for the region the model actually visits—clip before you extrapolate, rotate the box onto the ergodic cloud, smooth every kink the Jacobian sees, anchor the infeasible corners, and judge accuracy on the ergodic set. Each safeguard in the code maps to a specific failure mode in the theory above.

Where to read the code

Concept	File / function
Chebyshev recurrence, nodes	smolyak.py: chebyshev_basis_1d, _level_points
Nested levels, new points/degrees	smolyak.py: _new_points, _new_degrees
Smolyak combination rule	smolyak.py: _multi_indices, SmolyakGrid.__init__
Fit / eval (LU-cached collocation)	smolyak.py: SmolyakGrid.fit, .eval
Box mapping, clipping	smolyak.py: to_unit/from_unit, clip
PCA-rotated grid	rotated_grid.py: RotatedGrid
Gauss–Hermite quadrature	expectations_full.py: gauss_hermite_3d
Two-branch expectation	expectations_full.py: expect_pieces_full, _branch_pieces
Logistic default probability	expectations_full.py: default_prob
Euler residuals, μ closed form	time_iteration_full.py, period_map.py: mu_closed_form
Fischer–Burmeister smoothing	newton_collocation.py: _fb, _point_residual_fb
Outer fixed point, best iterate	time_iteration_full.py: time_iterate_full
Ergodic simulation / box adapt	simulate.py, simulate_full.py

References and further reading

- L. Bocola (2016), “The Pass-Through of Sovereign Risk,” *Journal of Political Economy* 124(4). The model and its global-solution method (§II.C, eqs. 11–12 for the default process).
- S. A. Smolyak (1963), “Quadrature and interpolation formulas for tensor products of certain classes of functions,” *Soviet Math. Dokl.* 4. The original sparse-grid construction.
- D. Krueger and F. Kubler (2004), “Computing equilibrium in OLG models with stochastic production,” *J. Economic Dynamics & Control* 28. Introduced Smolyak sparse grids to economics; the nested construction the code follows.
- K. Judd, L. Maliar, S. Maliar and R. Valero (2014), “Smolyak method for solving dynamic economic models: Lagrange interpolation, anisotropic grid and adaptive domain,” *J. Economic Dynamics & Control* 44. Anisotropic levels and the efficient implementation. <https://bfi.uchicago.edu/wp-content/uploads/Judd-Maliar-Valero-1.pdf>
- L. N. Trefethen (2013), *Approximation Theory and Approximation Practice*, SIAM. The definitive modern treatment of Chebyshev interpolation, the Runge phenomenon, Lebesgue constants and spectral convergence (Chebfun examples on Lebesgue constants: <https://www.chebfun.org/examples/approx/LebesgueConst.html>).
- J. Brumm and S. Scheidegger (2017), “Using adaptive sparse grids to solve high-dimensional dynamic models,” *Econometrica* 85. Adaptive/local sparse grids—the principled route past the global-basis corner problem of §4.
- Package internal notes: docs/bocola2016_replication.md (BGP derivation, stage-by-stage accuracy, the corner/convergence diagnostics summarized here).